GENAICH-010 · DSA COACH PROJECT

DSA Coach — Code Exec Agent

An AI-powered code review & feedback generation system that verifies correctness by actually running submitted code, not just reading it.

Team: Mind Matrix

Group Code: GENAICH-010

Members: Deep Sikha Baliyarsingh, Taniprava Sahoo, Arpita Panda

Project: DSA Coach — Feedback Generation

Domain: DSA

Assigned Component: Code Exec Agent

Overview

DSA Coach is an AI-powered Data Structures & Algorithms feedback assistant built with Python, Streamlit, FastAPI, and LangChain. Unlike a simple chatbot that reads code and guesses whether it's correct, DSA Coach's correctness verdicts are grounded in a **Code Exec Agent** — an autonomous LLM agent that designs its own test cases and actually executes submitted code in a sandbox before reporting a verdict.

Students submit a problem statement and their solution (typed, pasted, or uploaded as a file, notebook, or photo, in any language), and receive structured feedback: a correctness verdict, time/space complexity analysis, categorized issues, and a nudge toward a better approach — without the full solution being given away.

Generative AI **LLM Applications** **Prompt Engineering** **Agentic AI / Tool Calling** **AI-Enabled Full-Stack**

Project Objectives

1. Give students structured, actionable feedback on DSA code submissions.
2. Ground correctness verdicts in real code execution, not LLM guesswork.
3. Build a genuine agent: the LLM decides what tests to run and when to stop, rather than following a hardcoded pipeline.
4. Support code in any language and any submission format (typed, file upload, notebook, or photo).
5. Provide progressive, Socratic hints for students who are stuck, without leaking solutions.
6. Auto-generate high-signal edge-case test suites for a given problem.
7. Package the system as a real full-stack app: a FastAPI backend plus a Streamlit frontend, rate-limited, tested, and containerized.

Features

AI-Powered Code Review

- Submit a problem statement, constraints, and a solution in any language.
- Get a correctness verdict, Big-O time/space complexity, categorized issues (bug / edge case / style / inefficiency), strengths, and an overall score.
- Every verdict is tagged `sandbox_verified` so it's clear whether it was empirically checked or reasoning-only.

Code Exec Agent

- An autonomous tool-calling loop, not a fixed pipeline step.
- The LLM designs its own test cases (normal + edge cases) and decides whether/how many times to call the `execute_code` tool.
- Capped at a small step budget so it can't loop forever on a confused response.

Progressive Hint System

- Socratic tutor mode — hints escalate across 3 levels of specificity.
- Remembers prior hints in the session so level 2/3 build on level 1 instead of repeating it.
- Always ends with a guiding question instead of an answer.

Auto Test-Case Generation

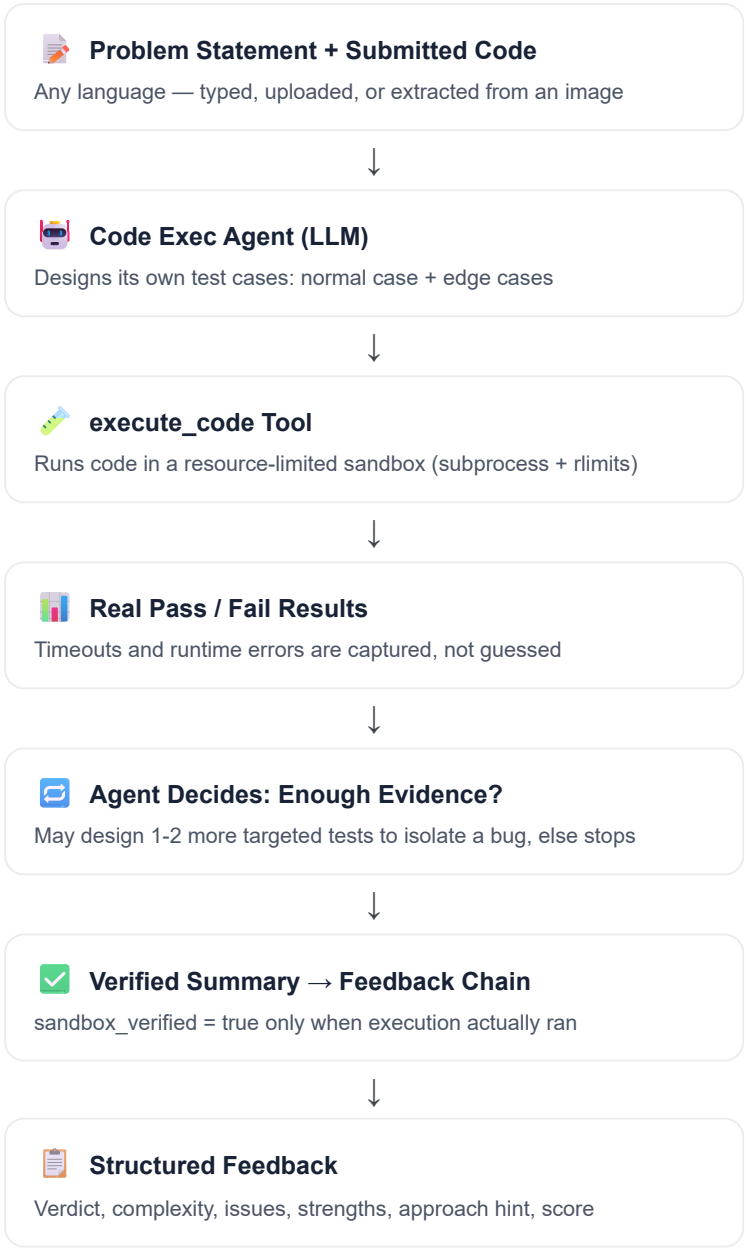
- Generates a high-signal edge-case suite for any problem statement.
- Covers empty/minimal input, boundary values, duplicates, and typical cases.

Multi-Format, Multi-Language Submission

- Upload a source file (`.py` , `.java` , `.js` , `.c` , `.cpp`), a Jupyter notebook (`.ipynb`), or a photo/screenshot of handwritten or typed code.
- Language auto-detected from file extension; photos are transcribed by the same vision-capable LLM already powering the coach — no separate OCR dependency.
- Feedback is generated regardless of language; sandbox execution runs for Python/JavaScript/C/C++ and falls back to careful reasoning-only review for others.

Code Exec Agent Workflow

The correctness check is implemented as a real agent loop, not a hardcoded call to a test runner. The engine hands the agent a problem and code, and a tool it is allowed to call — everything else (how many tests, which edge cases, when to stop) is the agent's own decision.



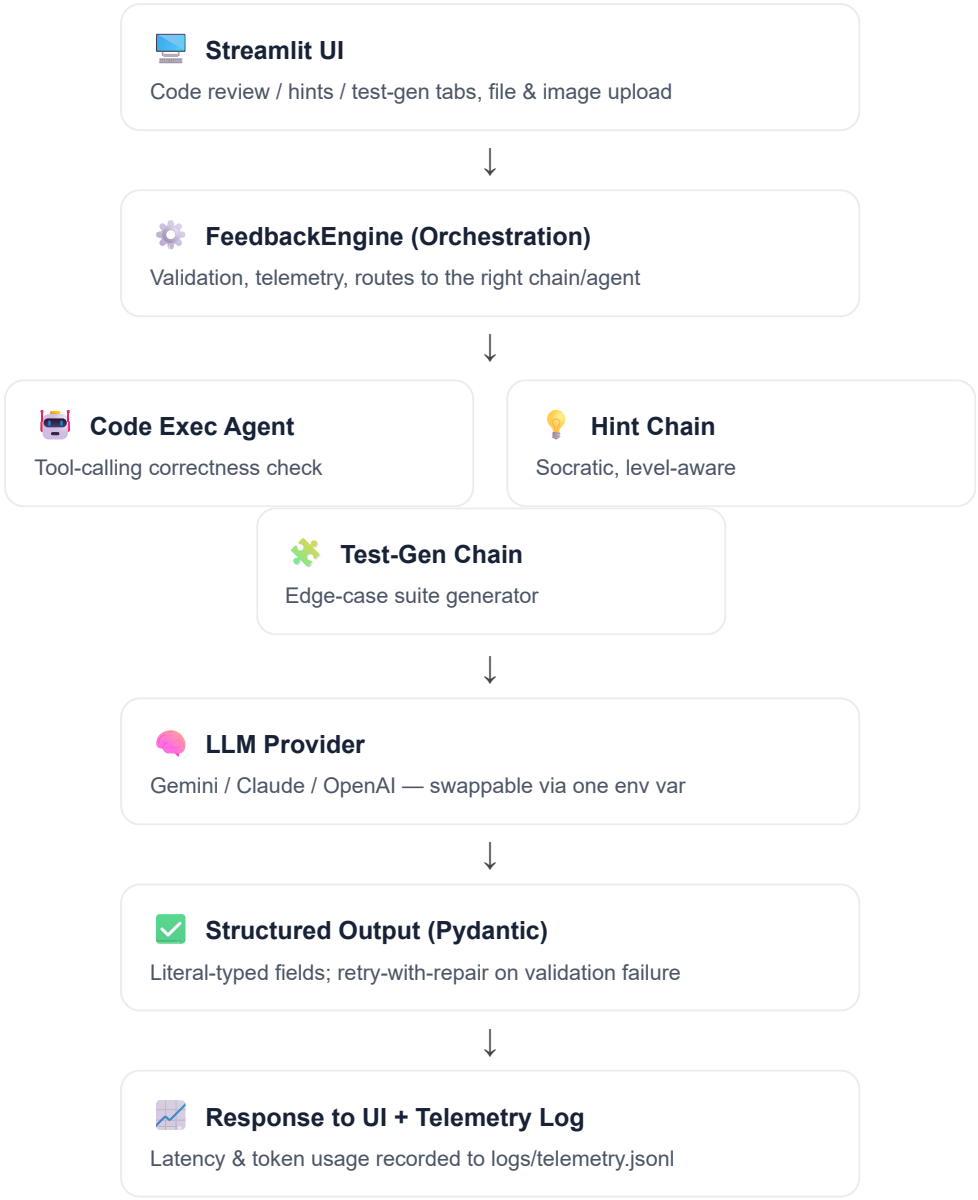
A step budget (`MAX_AGENT_STEPS = 4`) prevents a confused model from calling the tool indefinitely — after the budget is exhausted the agent is forced to produce a final summary instead of another tool call.

Example

> **Problem:** Two Sum — return indices of two numbers that add up to a target.
> **Agent trace:** designs 3 test cases (normal, empty-adjacent, no-solution case) → calls `execute_code` once → 3/3 passed → stops and reports `sandbox_verified: true` , verdict `correct` .

System Architecture

The frontend (Streamlit) never talks to the LLM directly — it goes through a `FeedbackEngine` orchestration layer, which is reachable either in-process (`embedded` mode) or over HTTP via a FastAPI backend (`api` mode), so the same core logic works as a single script or as a real deployed service.



Deployment Modes

Mode	How it runs
`embedded`	Streamlit imports the LangChain code directly — one process, simplest for local dev/demo.
`api`	Streamlit calls a separately-deployed FastAPI backend over HTTP; both containerized via `docker-compose.yml` for a genuine full-stack deployment.

Project Structure

```
dsa-coach/
├── app.py                        # Streamlit UI
├── dsa_coach/
│   ├── models.py           # Pydantic schemas
│   ├── prompts.py          # Prompt engineering
│   ├── chains.py           # LLM + structured output
│   ├── engine.py           # Orchestration layer
│   ├── code_exec_agent.py  # ★ Code Exec Agent
│   ├── tools.py            # execute_code tool
│   ├── sandbox.py          # Subprocess code execution
│   ├── code_extraction.py  # File/image/notebook ingest
│   ├── telemetry.py        # JSONL latency/token logs
│   └── client.py           # HTTP client for API mode
├── backend/
│   ├── main.py            # FastAPI service
│   └── rate_limit.py      # Per-IP rate limiter
├── evals/                  # Labeled dataset + accuracy script
├── tests/                  # pytest suite (no API key needed)
├── Dockerfile.backend
├── Dockerfile.frontend
├── docker-compose.yml
├── requirements*.txt
└── .env.example
```

Technologies Used

Technology	Purpose
Python	Backend and AI orchestration logic
Streamlit	Frontend user interface
LangChain	Prompt templates, structured output, tool-calling agent loop
Gemini / Claude / OpenAI	LLM reasoning, feedback generation, and vision-based code transcription (swappable via one env var)
Pydantic	Literal-typed structured output schemas
FastAPI	Rate-limited backend API for full-stack deployment
Subprocess + rlimits	Sandboxed multi-language code execution
Docker / docker-compose	Containerized backend + frontend deployment
pytest	Automated tests for models, sandbox, agent, and engine
Git/GitHub	Version control

Example Use Cases

Code Review
Submit a Two Sum solution in Python (typed or uploaded) → receive verdict `correct`, complexity `O(n) time, O(n) space`, 2 strengths, 0 bugs, sandbox-verified.

Stuck on a Problem
"I don't know how to avoid the O(n²) brute force." → Level 1 hint: "think about what information you could remember as you scan the array once." Level 2 builds further without repeating level 1.

Test-Case Generation
"Generate tests for: reverse a singly linked list." → suite covering empty list, single node, even/odd length, and a normal case, each with a rationale.

Uploaded Java File
Student uploads `Solution.java` → language auto-detected as Java → Code Exec Agent notes Java sandbox execution is unavailable in this environment and falls back to careful reasoning-only review, clearly labeled as such.

Photo of Handwritten Code
Student uploads a phone photo of code written on paper → vision-capable LLM transcribes it exactly → transcription shown for the student to confirm/edit → review proceeds normally.

Installation

Clone and set up

```
git clone <your-repository-url>
cd dsa-coach
python -m venv venv
venv\Scripts\activate          (Windows)   /   source venv/bin/activate   (Mac/Linux)
pip install -r requirements.txt
```

Configure environment variables

Copy `.env.example` to `.env` and set:

```
DSA_COACH_LLM_PROVIDER=google
DSA_COACH_MODEL=gemini-flash-latest
GOOGLE_API_KEY=your-key-here
DSA_COACH_MODE=embedded
```

Run

```
streamlit run app.py
```

Or, for the full-stack Docker deployment: `docker compose up --build`

Future Enhancements

- Add a JDK image to the backend Dockerfile to enable Java sandbox execution.
- Persist submission history per user (would need authentication).
- Swap local JSONL telemetry for a hosted tracing backend (e.g. LangSmith).
- True token-by-token streaming for feedback text.
- Difficulty-based problem recommendations and weak-topic tracking.
- Replace in-memory rate limiting with Redis for multi-instance deployment.
- Extend the Code Exec Agent with a self-correction loop: if the agent's own designed test case looks malformed, let it retry before calling the tool.

Team Project

This component — the Code Exec Agent — was built as part of the **DSA Coach — Feedback Generation** project by **Team Mind Matrix (GENAICH-010)**, combining generative AI, LLM-based agents, prompt engineering, sandboxed code execution, and full-stack deployment into a single working system.

Data Structures & Algorithms **Generative AI** **Agentic AI** **Prompt Engineering** **Full-Stack Deployment**

Project Summary

DSA Coach's Code Exec Agent turns code review from an LLM guessing game into a verifiable process: the agent designs its own tests, actually runs the student's code, and only then reports a verdict — with the UI always making clear whether that verdict was empirically checked or reasoning-only. Combined with progressive hints, automatic test-case generation, multi-language and multi-format submission support, and a real FastAPI + Streamlit full-stack deployment, it moves beyond a basic feedback chatbot into a genuinely agentic DSA coaching tool.